

```

positions = []
for i in range(len(seq) - k + 1):
    substring = seq[i:i+k]
    if substring == most_freq_substring:
        positions.append(i)

distances = []
for p1,p2 in zip(positions[:-1], positions[1:]):
    distances.append(p2 - p1)

print(distances)

```

5. † Du har fem nummer i en lista [13, 24, 42, 66, 78] och vill beräkna alla möjliga sätt ett nummer kan skrivas som en "direkt" sammanslagning av 2 element ur listan; med en "direkt sammanslagning" av två element (heltal) menas här t.ex. 13, 78 -> 1378 och 66, 42 -> 6642. Dvs, du vill ta reda på alla permutationer av två element ur listan så och slå samman varje permutation till ett tal.

Tips: Titta på standardmodulen `itertools` för permutationerna. För att slå samman heltal, försöka att först explicit typomvandla talen i listan till `str`, sen konkatenera permutationerna m.h.a strängfunktionen `.join()` och till sista typomvandla tillbaka till `int`.

Utdata bör bli (om alla tal sparats i en lista som man printar):

```
[1324, 1342, 1366, 1378, 2413, 2442, 2466, 2478, 4213, 4224, 4266, 4278, 6613,
6624, 6642, 6678, 7813, 7824, 7842, 7866]
```

6. Skriv om laboration 1 (temperaturkonvertering) så att funktionerna hamnar i separat modul med namnet `temp_functions.py` och dokumentera funktionerna med en dokumentationssträng (se kapitel 2). Huvudprogrammet ska importera funktionerna i `temp_functions.py` samt hantera fel med `try/except`.

Kapitel 8

Funktionell programmering

I detta kapitel ska vi titta på *funktionell* programmering i Python. Detta sätt att programmera på bygger på att man anropar, sätter ihop och modifiera funktioner. På detta sätt kan vi göra mycket av vad vi redan sett i de andra kapitlen, men endast genom att hantera funktioner. T.ex. kan vi byta ut loopar mot *rekursiva* funktioner, dvs funktioner som anropar sig själva.

8.1 Rekursion

Funktioner kan anropa sig själva och då kallas de *rekursiva*. Det finns många exempel på detta i matematiken, men även inom programmering kan rekursion vara användbart då det ofta leder till eleganta och korta program. Det kan dock vara svårt att börja “tänka rekursivt” och se hur man kan lösa olika problem med hjälp av rekursion. Ett bra sätt att lära sig använda rekursion är att ta program man skrivit med hjälp av loopar och göra om dem med rekursion istället.

8.1.1 Fakultet

Ett klassiskt exempel på en rekursiv funktion från matematiken är fakultet. Detta skrivs $n!$ och beräknas genom att man tar produkten av alla tal från 1 till n . Exempel:

$$5! = 5 * 4 * 3 * 2 * 1 = 120$$

Detta kan beskrivas rekursivt, dvs i termer av sig själv, genom:

$$\begin{aligned} 5! &= 5 * 4! \\ &= 5 * 4 * 3! \\ &= 5 * 4 * 3 * 2! \\ &= 5 * 4 * 3 * 2 * 1! \\ &= 5 * 4 * 3 * 2 * 1 * 0! \\ &= 5 * 4 * 3 * 2 * 1 * 1 \\ &= 120 \end{aligned}$$

Detta är en rekursiv beräkning med *basfall* $0! = 1$. En matematiker skriver detta som:

$$n! = \begin{cases} 1 & \text{om } n \text{ är } 0 \\ n * (n - 1)! & \text{annars} \end{cases}$$

I Python skriver vi detta genom:

```
def fac(n):  
    if n == 0:  
        return 1  
    else:  
        return n * fac(n - 1)
```

Fallet $n == 0$ kallas *basfall*. Det är här som rekursionen stannar och börjar nystas upp. Det andra fallet kallas *rekursivt steg* och går ut på att göra ett "enkla" anrop, där problemet i någon mening har gjorts mindre.

Denna funktion anropas precis på samma sätt som vanligt:

```
print(fac(5))
```

```
120
```

8.1.2 Fibonacci-talen

Varje kurs som tar upp rekursion måste ha med Fibonacci-talen som exempel. Detta är en talserie som ser ut på följande sätt:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Det n :te Fibonacci-talet, $F(n)$, är summan av de två tidigare Fibonacci-talen. Denna sekvens har två basfall, $F(0) = 0$ och $F(1) = 1$. Därför kan vi skriva

$$F(n) = \begin{cases} 0 & \text{om } n \text{ är } 0 \\ 1 & \text{om } n \text{ är } 1 \\ F(n - 1) + F(n - 2) & \text{annars} \end{cases}$$

Detta är tydligt en rekursiv definition och vi kan implementera den genom:

```
def fib(n):  
    if n == 0:  
        return 0  
    if n == 1:  
        return 1  
    else:  
        return fib(n-1) + fib(n-2)  
  
for i in range(10):  
    print(fib(i))
```

```
0  
1
```

```
1
2
3
5
8
13
21
34
```

Du kan jämföra denna implementation av en funktion som beräknar Fibonacci-tal med den i uppgift 6 i [Kapitel 6](#), som använder sig av en generator.

8.1.3 Största gemensamma delare, rekursivt

Minns [Euklides algoritmen](#) för största gemensamma delare (GCD, för "Greatest Common Divisor") för två tal a och b från avsnitt 3.3.3:

- Antag $a \geq b$,
- låt r vara resten av heltalsdivision $a // b$,
- om $r == 0$, då är $\text{GCD}(a, b) = b$,
- annars kan man använda att $\text{GCD}(a, b) = \text{GCD}(b, r)$.

En enkel iterativ implementation:

```
def gcd(a,b):
    if b > a:
        print("a must be bigger than b in gcd(a,b)")
        return None

    r = a % b
    while r != 0:
        a = b
        b = r
        r = a % b
    return b

print("GCD: " + str(gcd(15,12)) + "\n")
print("GCD: " + str(gcd(12,15)))
```

```
GCD: 3
```

```
a must be bigger than b in gcd(a,b)
```

```
GCD: None
```

Det är inte helt lätt att se hur denna implementation följer av algoritmen. En rekursiv implementation är mycket närmare:

```
def gcd(a,b):
    if b > a:
        print("a must be bigger than b in gcd(a,b)")
        return None
```

```

r = a % b
if r == 0:
    return b
else:
    return gcd(b,r)

print("GCD: " + str(gcd(15,12)) + "\n")
print("GCD: " + str(gcd(12,15)))

```

GCD: 3

a must be bigger than b in gcd(a,b)
GCD: None

8.1.4 Loopar genom rekursion

Ovan såg vi ett exempel på ett program vi redan skrivit med loopar som kan ersättas med rekursion. Detta kan alltid göras, både för **for** och **while** loopar.

I GCD-exemplet var det en **while** loop som byttes ut mot en rekursiv definition. Nedan är en enkel funktion som använder en **for** loop för att beräkna summan av alla tal upp till och med x:

```

def f(x):
    s = 0

    for i in range(x+1):
        s += i;

    return s

```

Detta kan skrivas rekursivt på följande sätt:

```

def f_rec(x):
    if x == 0:
        return 0
    else:
        return x + f_rec(x-1)

```

Tanken är här att vi anropar `f_rec` rekursivt med mindre och mindre tal tills vi kommer till basfallet när x är 0. Detta representerar loopen i f. Vi kan testa så att funktionerna returnerar samma resultat genom:

```

for i in range(5):
    print("f(" + str(i) + ") = " + str(f(i)))
    print("f_rec(" + str(i) + ") = " + str(f_rec(i)))

```

```

f(0) = 0
f_rec(0) = 0
f(1) = 1
f_rec(1) = 1
f(2) = 3
f_rec(2) = 3
f(3) = 6

```

```
f_rec(3) = 6
f(4) = 10
f_rec(4) = 10
```

8.1.5 Rekursion över listor

Vi har sett hur man kan skriva rekursiva funktioner över tal. Detta går även att göra över andra datatyper. Det generella tankesättet är att man måste ha ett eller fler basfall och att de rekursiva fallen reducerar indata på något sätt så att man till slut kommer till ett basfall. För tal använde vi ofta 0 som basfall och minskade indata när vi gjorde rekursiva anrop. Vill man skriva rekursiva funktioner över listor använder man istället ofta [] som basfall och tar bort ett element från början eller slutet i det rekursiva anropet.

Vi kan på detta sätt skriva vår egen len funktion genom:

```
def length(xs):
    # obs: lists behave as boolean values ([] is False)
    if xs:
        xs.pop()
        return (1 + length(xs))
    else:
        return 0

print(length([1,341,23,1,0,2]))
```

```
6
```

Varning: den här funktionen ändrar xs! Testa följande:

```
mylist = [1,2,3,6,7]

print(mylist)

print(length(mylist))

print(mylist)
```

```
[1, 2, 3, 6, 7]
5
[]
```

Då length har anropat pop upprepade gånger har mylist blivit tom! För att det inte ska bli så här måste vi anropa length med en *kopia* av mylist:

```
mylist = [1,2,3,6,7]

print(mylist)

print(length(mylist.copy()))

print(mylist)
```

```
[1, 2, 3, 6, 7]
5
[1, 2, 3, 6, 7]
```

Summan av alla element i en lista kan beräknas på följande sätt

```
def sum(xs):
    if xs:
        x = xs.pop() # pop returns the element
        return (x + sum(xs))
    else:
        return 0

print(sum([1,3,5,-1,0,2]))
```

```
10
```

och produkten på detta sätt:

```
def prod(xs):
    if xs:
        x = xs.pop() # pop returns the element
        return (x * prod(xs))
    else:
        return 1

print(prod([1,3,5,-1,2]))
```

```
-30
```

8.1.5.1 Sortering av listor

En snabb och bra sorteringsalgoritm är *quicksort*. Den fungerar på följande sätt:

1. Välj ett element i listan ("pivotelementet").
2. Dela upp listan i två dellistor: en med alla element som är mindre än pivotelementet och en med alla element som är större än pivotelementet.
3. Sortera dellistorna genom rekursion. Basfallet är tomma listan vilken redan är sorterad.
4. Sätt ihop listorna med pivotelementet i mitten.

För att implementera detta i Python skriver vi först två hjälpfunktioner för att hämta ut alla element som är mindre och större än ett givet element:

```
# All elements less than or equal to v in xs
def smallereq(xs,v):
    res = []
    while xs:
        x = xs.pop()
        if x <= v:
            res.append(x)
    return res
```

```
# All elements greater than v in xs
```

```
def greater(xs,v):  
    res = []  
    while xs:  
        x = xs.pop()  
        if x > v:  
            res.append(x)  
    return res
```

```
print(smaller([2,4,5,6,7,8],5))  
print(greater([2,4,5,6,7,8],5))
```

```
[5, 4, 2]  
[8, 7, 6]
```

Vi kan sedan implementera quicksort på följande sätt:

```
def quicksort(xs):  
    if xs == []:  
        return xs  
  
    p = xs.pop()  
    s = quicksort(smaller(xs.copy(),p))  
    g = quicksort(greater(xs.copy(),p))  
    return (s + [p] + g)
```

```
print(quicksort([8,3,3,2,4,5,8,5,6,7,8]))
```

```
[2, 3, 3, 4, 5, 5, 6, 7, 8, 8, 8]
```

Vi kan även skriva smaller och greater direkt med listomfattningar och få en ännu kortare implementation:

```
def quicksort(xs):  
    if xs == []:  
        return xs  
    else:  
        p = xs.pop()  
        s = quicksort([ x for x in xs if x <= p ])  
        g = quicksort([ x for x in xs if x > p ])  
        return (s + [p] + g)
```

```
print(quicksort([8,3,3,2,4,5,8,5,6,7,8]))
```

```
[2, 3, 3, 4, 5, 5, 6, 7, 8, 8, 8]
```

8.1.6 Effektiv rekursion

En fara med att skriva rekursiva funktioner är att de kan bli väldigt långsamma jämfört med iterativ kod skriven med loopar. Exempelvis om vi utvidgar definitionen av fib(5) för hand så ser vi följande:


```

fib(5) = fib(4) + fib(3)
       = (fib(3) + fib(2)) + (fib(2) + fib(1))
       = ((fib(2) + fib(1)) + (fib(1) + fib(0))) + ((fib(1) + fib(0)) + fib(1))
       = (((fib(1) + fib(0)) + fib(1)) + (fib(1) + fib(0)))
         + ((fib(1) + fib(0)) + fib(1))
       = (((1 + 0) + 1) + (1 + 0)) + ((1 + 0) + 1)
       = 5

```

Problemet är att vi beräknar samma tal många gånger, så vi får väldigt långsam kod. Testar man följande så ser man att det tar längre och längre tid att skriva ut utdata:

```

for x in range(35):
    print(fib(x))

```

```

0
1
1
2
3
5
8
13
21
34
55
89
144
233
377
610
987
1597
2584
4181
6765
10946
17711
28657
46368
75025
121393
196418
317811
514229
832040
1346269
2178309
3524578
5702887

```

Det sista talet här tar flera sekunder att skriva ut.

Detta kan lösas genom att skriva en smartare version av fib:

```
def fib_rec(n, a=0, b=1):
    if n == 0:
        return a
    elif n == 1:
        return b
    else:
        return fib_rec(n-1, b, a+b)
```

Tanken här är att n är en räknare för indexet på det Fibonacci tal vi håller på att beräkna, a är värdet 2 steg tidigare i sekvensen och b är värdet 1 steg tidigare. På detta sätt behöver vi bara ett rekursivt anrop och koden blir mycket snabbare. Exempelvis skrivs följande ut på nolltid:

```
print(fib_rec(100))
```

```
354224848179261915075
```

Denna funktion kommer fungera på följande sätt för att räkna ut fib_rec(5) (dvs. fib_rec(5,0,1)):

```
fib_rec(5) = fib_rec(5,0,1)
           = fib_rec(4,1,1)
           = fib_rec(3,1,2)
           = fib_rec(2,2,3)
           = fib_rec(1,3,5)
           = 5
```

På detta sätt har vi alltså undvikit den kombinatoriska explosionen och fib_rec är väldigt mycket snabbare än fib.

Vill man prova med större tal går det också:

```
print(fib_rec(1000))
```

```
434665576869374564356885276750406258025646605173717804024817290895365554179490518904
038798400792551692959225930803226347752096896232398733224711616429964409065331879382
98969649928516003704476137795166849228875
```

Vill man prova med väldigt stora tal måste man göra följande då Python har en inbyggd gräns för hur många rekursiva anrop som tillåts:

```
import sys

sys.setrecursionlimit(100000)

print(fib_rec(10000))
```

Beräkningen av detta väldigt stora tal går på nolltid, även fast det består av över 2000 siffror. Så rekursiva funktioner kan vara väldigt snabba om man skriver dem på rätt sätt.

Python är dock inte alltid jättesnabbt när det kommer till att exekvera funktioner skrivna med rekursion. Så det är oftast bättre att skriva en funktion med hjälp av for eller while om man behöver snabb kod.

8.1.7 Diskussion om rekursion

- Princip: bryt upp i mindre instanser, lös dessa.
- Rekursion är att “ta hjälp av en (osynlig) kompis”
- Rekursion är en effektiv algoritmisk teknik
 - Rekursion är dock inte Pythons starkaste sida. Många programmeringsspråk konverterar internt (utan att programmeraren behöver göra något) viss rekursiv kod till while-loopar. Det är möjligt när det rekursiva anropet är sist i funktionen, vilket kallas “svansrekursion”, och ger snabbare och mer minnessnål exekvering.
- Ofta användbar teknik i datastrukturer (t.ex. sökträd).

Principen för ett funktionsanrop är som följer.

- När ett funktionsanrop görs läggs anropsparametrarna på den minnesarea som kallas *stacken*.
- En funktion börjar exekvera genom att plocka upp anropsparametrarna från stacken till lokala variabler.
- När **return** exekveras så läggs returvärdet på stacken.
- Den kod som gjort ett funktionsanrop kan alltså hämta resultatet på stacken när funktionen är klar.

Det är denna princip som gör att rekursion i praktiken är enkel i datorer.

8.2 Anonyma och högre ordningens funktioner

När vi har pratat variabler och värden har vi hittills framförallt pratat om heltal, flyttal, strängar, listor, med mera. Det finns goda skäl att lägga *funktioner* till uppräknningen av värden att arbeta med, vilket handlar om möjligheten att skapa nya funktioner när man behöver dem, skicka funktioner som argument, returnera funktioner, och kunna tillämpa operatorer på funktioner. När ett programmeringsspråk har de möjligheterna säger man att funktioner behandlas som *first class citizens* (eller *first class objects* eller *first class functions*).

Gamla språk i samma tradition som Python, så kallade *imperativa språk*, var dåliga på att hantera funktioner, men i Lisp och andra språk för *funktionell programmering* så har funktioner varit *first class citizens* sedan 50-talet. Idag kan man säga att det är en vanlig finess som många språk stödjer, däribland Python.

8.2.1 Anonyma funktioner

Begreppet *anonyma funktioner* med hjälp av *lambda-uttryck* finns i många språk idag. Man säger att funktionerna är *anonyma* eftersom man inte ger dem ett namn, utan bara skapar dem. Tag kodsnutten

```
double = lambda x: 2 * x

print(double(3))
```

6

Definitionen `double` är en funktion som tar ett argument, `x`, och beräknar `2 * x`. Vi kan även få en funktion av två argument genom:

```
f = lambda x, y: 2 * x + y

print(f(3,4))
```

Detta är ekvivalent med följande definition:

```
def f(x,y):
    return 2 * x + y
```

Man ska dock inte se lambda-uttryck som bara ett alternativ till vanliga funktionsdefinitioner; de är snarare komplement till **def** och dess främsta användningsområde är för att skapa tillfälliga små funktioner att använda tillsammans med *högre ordningens funktioner*.

Anledningen att dessa introduceras med **lambda** är från den så kallade λ -kalkylen. Detta är en beräkningsmodell uppfunnen av logikern Alonzo Church på 1930-talet (https://en.wikipedia.org/wiki/Lambda_calculus). I denna modell är allting uppbyggt från funktioner och variabler (så t.ex. heltal representeras även de som funktioner), och man kan på detta sätt representera alla beräkningar som kan göras med en **Turingmaskin** genom att bara använda lambda-uttryck.

Anmärkning: En Turingmaskin är en matematisk beräkningsmodell/koncept, som används för att förstå algoritmer och deras begränsningar. Datorer som används idag är inte uppbyggda enligt Turingmaskinkonceptet, men termen Turingkomplett om en formell definition av regler, som t.ex. utgör ett programmeringsspråk, är sådan att vilken Turingmaskin som helst kan representeras med instruktioner enligt dessa regler.

8.2.2 Högre ordningens funktioner

En *högre ordningens funktion* tar en funktion som argument och är ett djupare begrepp än det kanske först låter som. Poängen med högre ordningens funktioner är att de kan vara ett sätt att generalisera funktionalitet. Genom att identifiera vanliga mönster för beräkningar och låta dessa implementeras med högre ordningens funktioner kan man förenkla, korta ner, och kanske även snabba upp sin kod. Några exempel på beräkningar man kan vilja göra är:

1. Ta en lista med tal och returnera en lista med dess kvadrater.
2. Ta en lista med icke-tomma listor och returnera en lista med det första elementet från varje lista.
3. Ta en lista med strängar och räkna ut deras längder.

Dessa tre beräkningar har gemensamt att man ska utföra något på varje element i en lista. Det är inte svårt att skriva en **for**-loop för detta, men funktionen `map` är mer kompakt, känns igen av andra programmerare, och är redan implementerad (se också uppgift 7 i [kapitel 6](#)). Det första argumentet till `map` är en funktion som sedan appliceras på alla element i listan. Denna funktion kan antingen ges genom `lambda` eller genom namn på vilken Pythonfunktion som helst.

Ovan uppgifter kan därför lätt lösas med några små enkla uttryck:

```
# 1
print(list(map(lambda x: x**2, [1,2,3,4])))

# 2
print(list(map(lambda t: t[0], [[1,1], [0,2,1], [3,1,53,2]])))

# 3
print(list(map(len, ["hej", "hopp"])))
```

```
[1, 4, 9, 16]
[1, 0, 3]
[3, 4]
```

Obs: map returnerar en generator, så vi måste lägga in `list` för att få ut en lista.

8.2.2.1 Inte bara för listor

Observera också att map och liknande funktioner inte är låsta till listor, utan kan tillämpas på allt som i Python är definierat som *itererbart*. Det inkluderar till exempel strängar, tupler och uppslagstabeller, men kan också definieras av programmeraren själv.

För att beräkna längden på varje rad i filen `people.txt` med innehåll:

```
Anders
Christian
Kristoffer
Lars
```

kan vi köra:

```
print(list(map(len, open('people.txt'))))
```

```
[7, 10, 11, 5]
```

Notera att newline tecknet `\n` räknas med och är av längd 1.

8.2.2.2 Högre ordningens funktion: filter

En annan typ av funktionalitet som vi ofta vill använda är att välja ut element som uppfyller något kriterium. Till det finns funktionen `filter`. Den tar ett predikat, dvs en funktion som returnerar sant eller falskt, och en itererbar datastruktur och returnerar en generator (som du kan iterera över eller plocka fram som en lista med `list`) med de element som predikatet är sant för (`True` eller motsvarande, tex icke-noll). Exempel:

```
print(list(filter(lambda x: x % 2, range(10))))

print(list(filter(lambda x: x, [[1], [2,3], [], [], [4,5,6]])))
```

```
[1, 3, 5, 7, 9]
[[1], [2, 3], [4, 5, 6]]
```

8.2.3 Egna högre ordningens funktioner

Funktionerna `map` och `filter` har ingen särställning i Python, de är helt enkelt bra verktyg som många finner praktiska att använda. Det är inte så svårt att skriva dessa funktioner själv. Däremot kan det ju vara så att du hittar egna beräkningsbehov som kan generaliseras.

Till exempel kan man beräkna summan och produkten av elementen i två listor med hjälp av slicing på följande sätt:

```
def sum_rec(xs):
    if xs == []:
        return 0
```

```

    else:
        return (xs[0] + sum_rec(xs[1:]))

def prod_rec(xs):
    if xs == []:
        return 1
    else:
        return (xs[0] * prod_rec(xs[1:]))

print(sum_rec([1,2,3,4]))
print(prod_rec([1,2,3,4]))

```

```

10
24

```

Den enda skillnaden mellan dessa är att en använder + och 0, och den andra * och 1. Vi kan alltså generalisera dessa till en funktion som tar in en funktion f och ett element x :

```

def foldr(xs,f,x):
    if xs == []:
        return x
    else:
        return f(xs[0], foldr(xs[1:], f, x))

sum = lambda xs: foldr(xs, lambda x,y: x + y, 0)
prod = lambda xs: foldr(xs, lambda x,y: x * y, 1)

print(sum([1,2,3,4]))
print(prod([1,2,3,4]))

```

```

10
24

```

Vi kan nu även lätt skriva en funktion som använder exponentiering som operation:

```

exps = lambda xs: foldr(xs, lambda x,y: x ** y, 1)

print(exps([2,3,4]))

```

```

2417851639229258349412352

```

Vi kan även skriva en funktion för att slå ihop listor av listor:

```

join = lambda xs: foldr(xs, lambda x,y: x + y, [])

print(join([[1], [2,3], [], [], [4,5,6]]))

```

```

[1, 2, 3, 4, 5, 6]

```

Högre ordningens funktioner är alltså mycket smidiga för att skriva väldigt generella funktioner och minimera duplikation av kod!

Obs: det finns en funktion som heter reduce som fungerar exakt som foldr, men med argumenten i en

annan ordning. Den ligger i biblioteket `functools` och importeras på följande sätt:

```
import functools

print(functools.reduce(lambda s, x: s+x, [1,2,3], 0))
```

6

8.3 Uppgifter

1. Skriv en rekursiv funktion `rec_prod_m(n,m)` som tar två tal n och m och beräknar $n * (n-m) * (n-2m) * \dots$ tills $n-km$ har blivit negativt.

Tips: Funktionen kan skrivas genom att modifiera funktionen `fac` något.

2. Vad händer om vi ändrar till `return 0` i basfallet för `prod`? Varför?
3. Deklarera fem funktioner `f1, ..., f5` med hjälp av `lambda` som utför addition, subtraktion, multiplikation, division, och exponentiering. Till exempel ska `f1(4,7)` returnera 11. Använd dessa funktioner för att skriva uttrycken

```
(6*(2+3)/5)**2
(10 - 2**3)*5
```

4. Använd `map` och `lambda` för att skriva funktion `get_evens(xs)` som tar en lista med tal `xs` och konverterar jämna tal till `True` och udda tal till `False`.
5. Studera koden nedan. Hur många anrop till `f` görs totalt om vi anropar `f` med 3? Hur många blir det för 4? Varför?

```
def f(n):
    if n <= 1:
        return 1
    else:
        return f(n-1) + f(n-2) + f(n-3)
```

6. Collatz-sekvensen med start i n får man med följande regler:

- Om $n = 1$ är vi klara.
- Om n är jämnt, fortsätt med $n/2$.
- Annars, fortsätt med $3n + 1$.

Collatz-sekvensen med start i $n = 10$ är: 10, 5, 16, 8, 4, 2. Om man startar med $n = 9$ får man en längre sekvens: 9, 28, 14, 7, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2.

Skriv en rekursiv funktion `collatz(n)` returnerar en lista med Collatz-sekvensen med start i n .

Exempel:

```
>>> collatz(10)
```

```
>>> collatz(10)
[10, 5, 16, 8, 4, 2]
```

7. Använd funktionen `foldr` för att skriva en funktion `joinstrings` som konkatenerar en lista med strängar.

8. Skriv `fac` och `fib` med hjälp av loopar istället.
9. Vad blir fel om vi inte kopierar `xs` i quicksort?
10. † Pascals triangel ser ut så här:

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

osv... Skriv en funktion `pascal(n)` som returnerar den `n`:e raden. Den ska fungera på följande sätt:

```
for i in range(1,8):
    print(pascal(i))
```

```
[1]
[1, 1]
[1, 2, 1]
[1, 3, 3, 1]
[1, 4, 6, 4, 1]
[1, 5, 10, 10, 5, 1]
[1, 6, 15, 20, 15, 6, 1]
```